19

A social network dataset is a categorical dataset to determine whether a user purchased a particular product based on gender, age and estimated salary.

Objectives:

1. Understand the Dataset & cleanup (if required).
2. Use a SVM to classify whether a user purchased a car or not? (Use Linear Kernel)
3. Create a confusion matrix and evaluate the model using accuracy

```
import pandas as pd
# Load Dataset
df = pd.read_csv('social_network.csv')
print(df.head())
```

```
   Unnamed: 0  Gender  Age  EstimatedSalary  Purchased
0           0    Male   49           131217          1
1           1  Female   56            92991          1
2           2    Male   49            24014          0
3           3    Male   21            31093          0
4           4    Male   47            38070          0
```

```
df.drop(columns=['Unnamed: 0'], inplace=True)
print(df.head())
```

```
   Gender  Age  EstimatedSalary  Purchased
0    Male   49           131217          1
1  Female   56            92991          1
2    Male   49            24014          0
3    Male   21            31093          0
4    Male   47            38070          0
```

```
# Data Cleaning
df.isnull().sum()
```

```
Gender             0
Age                0
EstimatedSalary    0
Purchased          0
dtype: int64
```

```
from sklearn.preprocessing import LabelEncoder
le = LabelEncoder()
df['Gender']=le.fit_transform(df['Gender'])
```

```
df.head()
```

```
   Gender  Age  EstimatedSalary  Purchased
0       1   49           131217          1
1       0   56            92991          1
2       1   49            24014          0
3       1   21            31093          0
4       1   47            38070          0
```

```
X = df[['Gender', 'Age', 'EstimatedSalary']]
y = df['Purchased']

from sklearn.model_selection import train_test_split
X_train, X_test, y_train, y_test = train_test_split(X, y,
test_size=0.2, random_state=42)

print('X_train :\n', X_train)
```

```
X_train :
      Gender  Age  EstimatedSalary
79         0   26            74021
197        0   20            38071
38         1   54            90031
24         0   45            49592
122        0   20            40056
..       ...  ...              ...
106        0   59            28308
14         0   39           125864
92         0   24            37019
179        1   23           120793
102        0   42           132566

[160 rows x 3 columns]
```

```
print('X_train shape:\n', X_train.shape)
```

```
X_train shape:
 (160, 3)
```

```
print('X_test :\n', X_test.head())
print('X_test shape:\n', X_test.shape)
```

```
X_test :
      Gender  Age  EstimatedSalary
95         0   57           111794
15         1   45            98069
30         1   30           109912
158        1   39           141305
128        0   25            33500
X_test shape:
 (40, 3)
```

```
from sklearn.preprocessing import StandardScaler

# Feature scaling
sc = StandardScaler()
X_train = sc.fit_transform(X_train)
X_test = sc.transform(X_test)

X_train.shape, X_test.shape
```

```
((160, 3), (40, 3))
```

```
X_train
```

```
array([[-1.03823026, -1.11311992, -0.29563699],
       [-1.03823026, -1.59017131, -1.26217344],
       [ 0.96317747,  1.11311992,  0.13480109],
       [-1.03823026,  0.39754283, -0.95242472],
       [-1.03823026, -1.59017131, -1.20880557],
       [-1.03823026,  1.51066275, -0.11095996],
       [-1.03823026,  0.31803426,  1.4147427 ],
       [ 0.96317747,  0.39754283, -1.59205946],
       [-1.03823026,  1.43115418,  0.52821772],
       [-1.03823026, -0.31803426, -0.67975121],
       [ 0.96317747, -1.43115418,  0.04301373],
       [-1.03823026, -1.51066275,  0.09054743],
       [-1.03823026,  1.11311992, -0.78613743],
       [-1.03823026,  0.07950857,  1.24939641],
       [-1.03823026, -1.11311992,  1.6322739 ],
       [-1.03823026,  0.87459422,  1.48843606],
       [-1.03823026, -1.66967988,  0.10821125],
       [-1.03823026,  1.35164562,  0.47675873],
       [ 0.96317747, -0.31803426,  0.87006782],
       [-1.03823026,  1.27213705,  0.58220395],
       [-1.03823026,  0.39754283, -1.12833705],
       [-1.03823026, -0.39754283,  0.23739644],
       [ 0.96317747,  1.43115418,  1.22624793],
       [ 0.96317747,  0.71557709, -1.00192107],
       [ 0.96317747, -1.19262848,  0.33762587],
       [ 0.96317747,  0.95410279, -1.53818077],
       [-1.03823026, -0.63606853, -0.24872166],
       [-1.03823026,  0.71557709,  1.05297039],
       [-1.03823026, -0.2385257 ,  0.79201899],
       [ 0.96317747,  0.15901713,  0.23253015],
       [-1.03823026,  1.11311992,  1.44689785],
       [ 0.96317747, -0.95410279, -1.65814421],
       [-1.03823026, -1.19262848,  0.71864825],
       [ 0.96317747,  0.07950857,  0.61581092],
       [-1.03823026,  1.43115418, -0.50658122],
       [-1.03823026, -0.87459422,  1.38016785],
       [-1.03823026,  1.27213705, -1.51368801],
```

```
       [ 0.96317747, -0.39754283,  0.0590913 ],
       [ 0.96317747, -0.95410279, -0.98847828],
       [ 0.96317747, -1.66967988,  0.94685302],
       [ 0.96317747,  0.71557709, -1.64010398],
       [-1.03823026, -0.55655996,  0.59484018],
       [-1.03823026,  1.43115418,  0.36671606],
       [ 0.96317747,  0.79508566, -0.35021471],
       [ 0.96317747,  0.79508566,  0.7981489 ],
       [ 0.96317747,  1.03361135,  0.42266495],
       [ 0.96317747,  1.19262848,  0.79960072],
       [-1.03823026,  0.31803426, -1.56909918],
       [ 0.96317747,  0.        , -0.9151882 ],
       [ 0.96317747, -0.31803426, -0.91392458],
       [-1.03823026,  0.55655996,  1.05587403],
       [ 0.96317747,  0.95410279, -1.39087469],
       [-1.03823026,  0.39754283,  1.0341236 ],
       [-1.03823026,  1.27213705,  1.66631103],
       [ 0.96317747, -0.79508566,  0.03502871],
       [ 0.96317747,  0.15901713,  0.62390348],
       [-1.03823026,  0.55655996, -0.80038679],
       [ 0.96317747,  0.        , -0.21667405],
       [-1.03823026,  0.79508566,  0.64186305],
       [-1.03823026, -0.31803426, -1.60722292],
       [ 0.96317747,  0.55655996, -1.26220033],
       [-1.03823026, -1.59017131,  0.14246347],
       [-1.03823026, -0.55655996,  0.2089515 ],
       [ 0.96317747,  0.79508566, -0.09754406],
       [ 0.96317747, -0.55655996, -1.71710427],
       [-1.03823026,  1.03361135,  0.76491833],
       [ 0.96317747,  1.03361135, -0.53771471],
       [ 0.96317747, -1.66967988,  1.16129238],
       [-1.03823026, -1.74918844,  0.26140526],
       [ 0.96317747, -1.51066275, -1.49435728],
       [-1.03823026, -0.15901713,  1.01463156],
       [ 0.96317747,  0.71557709,  1.24211042],
       [ 0.96317747,  1.35164562, -0.15438017],
       [ 0.96317747, -1.59017131, -0.12405324],
       [-1.03823026,  1.19262848,  1.30703908],
       [ 0.96317747, -0.2385257 , -0.5235729 ],
       [ 0.96317747,  0.71557709,  1.68830344],
       [-1.03823026,  0.71557709, -1.38767531],
       [-1.03823026,  0.31803426, -0.06089902],
       [-1.03823026, -1.74918844,  0.60567506],
       [ 0.96317747,  1.51066275,  0.70079623],
       [-1.03823026, -1.27213705, -0.35024159],
       [-1.03823026,  0.79508566,  0.81780226],
       [ 0.96317747, -0.87459422, -0.56723507],
       [-1.03823026,  0.15901713, -1.46177196],
       [-1.03823026, -0.95410279, -0.37704651],
```

```
       [ 0.96317747,  0.47705139, -1.30806712],
       [ 0.96317747,  0.79508566, -0.28706049],
       [-1.03823026, -0.55655996, -0.17965261],
       [ 0.96317747,  0.79508566,  0.306815  ],
       [ 0.96317747,  0.79508566,  1.05305105],
       [ 0.96317747, -0.87459422,  1.49456598],
       [-1.03823026,  0.79508566,  0.49996098],
       [ 0.96317747, -0.47705139, -0.96500717],
       [-1.03823026,  0.71557709,  1.58702547],
       [ 0.96317747, -0.39754283,  1.44966706],
       [ 0.96317747, -1.51066275, -1.44978099],
       [-1.03823026,  0.55655996, -0.88185008],
       [-1.03823026, -1.66967988, -0.32521112],
       [ 0.96317747,  0.95410279,  1.30158131],
       [ 0.96317747,  0.95410279, -1.48392567],
       [ 0.96317747,  0.47705139,  1.21546681],
       [ 0.96317747,  0.63606853, -1.62405329],
       [-1.03823026, -1.66967988,  1.6089641 ],
       [ 0.96317747, -1.35164562,  0.03750219],
       [-1.03823026, -1.35164562, -1.68435764],
       [ 0.96317747,  1.27213705,  0.45858408],
       [-1.03823026,  0.2385257 ,  0.65769865],
       [-1.03823026,  0.71557709,  1.01498107],
       [-1.03823026, -1.59017131, -0.8851839 ],
       [ 0.96317747,  0.39754283,  0.01075104],
       [ 0.96317747,  1.43115418, -0.13058644],
       [-1.03823026, -0.31803426,  1.07773201],
       [ 0.96317747, -0.63606853, -1.45956734],
       [ 0.96317747, -1.27213705,  1.1126026 ],
       [ 0.96317747,  0.79508566,  0.85517321],
       [ 0.96317747, -1.43115418,  0.88988249],
       [ 0.96317747,  1.35164562, -1.48825425],
       [-1.03823026,  1.51066275,  1.08867444],
       [ 0.96317747, -1.43115418, -1.22353887],
       [-1.03823026, -0.47705139, -0.66835173],
       [ 0.96317747,  0.15901713,  1.52373683],
       [ 0.96317747,  0.55655996, -1.03162963],
       [ 0.96317747,  0.07950857, -1.49153429],
       [ 0.96317747,  0.2385257 ,  0.14238282],
       [ 0.96317747, -1.66967988, -0.41111053],
       [-1.03823026,  0.95410279, -1.58808039],
       [ 0.96317747, -1.59017131,  0.91787037],
       [-1.03823026,  0.63606853,  0.03683005],
       [ 0.96317747,  0.71557709, -0.25294269],
       [-1.03823026,  1.27213705, -1.45319546],
       [-1.03823026, -1.59017131,  0.22868552],
       [ 0.96317747, -0.2385257 , -0.4121053 ],
       [ 0.96317747,  1.11311992, -1.13833848],
       [ 0.96317747,  1.19262848,  1.50975633],
```

```
       [ 0.96317747, -1.59017131, -0.42753762],
       [-1.03823026,  1.03361135, -0.31711856],
       [ 0.96317747, -1.11311992,  1.11467279],
       [-1.03823026, -0.07950857,  1.17607944],
       [ 0.96317747, -0.39754283,  0.69875293],
       [-1.03823026,  1.27213705,  0.21438239],
       [ 0.96317747, -0.47705139,  0.77513485],
       [-1.03823026, -0.15901713,  0.37142104],
       [-1.03823026,  1.19262848, -0.95825889],
       [ 0.96317747, -1.19262848,  0.83248178],
       [-1.03823026,  0.        , -1.05047642],
       [ 0.96317747, -1.74918844,  0.39298327],
       [ 0.96317747,  0.71557709, -1.29376399],
       [ 0.96317747,  0.2385257 ,  0.20427341],
       [ 0.96317747, -1.35164562, -1.66599479],
       [ 0.96317747, -0.55655996,  1.35045929],
       [ 0.96317747,  0.07950857,  0.19765956],
       [-1.03823026,  0.39754283, -1.23208848],
       [ 0.96317747, -0.15901713, -0.19728955],
       [-1.03823026,  0.07950857,  0.40645294],
       [-1.03823026,  1.51066275, -1.52465732],
       [-1.03823026, -0.07950857,  1.09819193],
       [-1.03823026, -1.27213705, -1.29045707],
       [ 0.96317747, -1.35164562,  0.96185517],
       [-1.03823026,  0.15901713,  1.27837906]])
```

```
from sklearn.svm import SVC

svm_model = SVC(kernel='linear', random_state=42) # to use RBF replace
with 'rbf'
svm_model.fit(X_train, y_train)
```

```
SVC(kernel='linear', random_state=42)
```

```
y_pred = svm_model.predict(X_test)

from sklearn.metrics import confusion_matrix, accuracy_score,
precision_score, recall_score, f1_score

# Confusion Matrix and Accuracy
cm = confusion_matrix(y_test, y_pred)
acc = accuracy_score(y_test, y_pred)
prec = precision_score(y_test, y_pred)
rec = recall_score(y_test, y_pred)
f1 = f1_score(y_test, y_pred)


print("\nConfusion Matrix:")
print(cm)
print(f"\n Model Accuracy: {acc*100:.2f}%")
```

```python
print(f"\n Model Precision: {prec*100:.2f}%")
print(f"\n Model Recall: {rec*100:.2f}%")
```

```
Confusion Matrix:
[[ 6  5]
 [ 6 23]]

 Model Accuracy: 72.50%

 Model Precision: 82.14%

 Model Recall: 79.31%
```

```python
import numpy as np
# Compute Confusion Matrix manually
TP = TN = FP = FN = 0
for actual, pred in zip(y_test, y_pred):
    if actual == 1 and pred == 1:
        TP += 1
    elif actual == 0 and pred == 0:
        TN += 1
    elif actual == 0 and pred == 1:
        FP += 1
    elif actual == 1 and pred == 0:
        FN += 1

cm = np.array([[TN, FP],
               [FN, TP]])

accuracy = (TP + TN) / (TP + TN + FP + FN)

print("\n Confusion Matrix (Manual):")
print(cm)
print(f"\n Accuracy (Manual): {accuracy * 100:.2f}%")
```

```
 Confusion Matrix (Manual):
[[ 6  5]
 [ 6 23]]

 Accuracy (Manual): 72.50%
```

- **Accuracy:**

$$\frac{TP + TN}{TP + TN + FP + FN}$$

- **Precision:**

$$\frac{TP}{TP + FP}$$

→ How many predicted positives are correct

- **Recall (Sensitivity):**

$$\frac{TP}{TP + FN}$$

→ How many actual positives are correctly detected

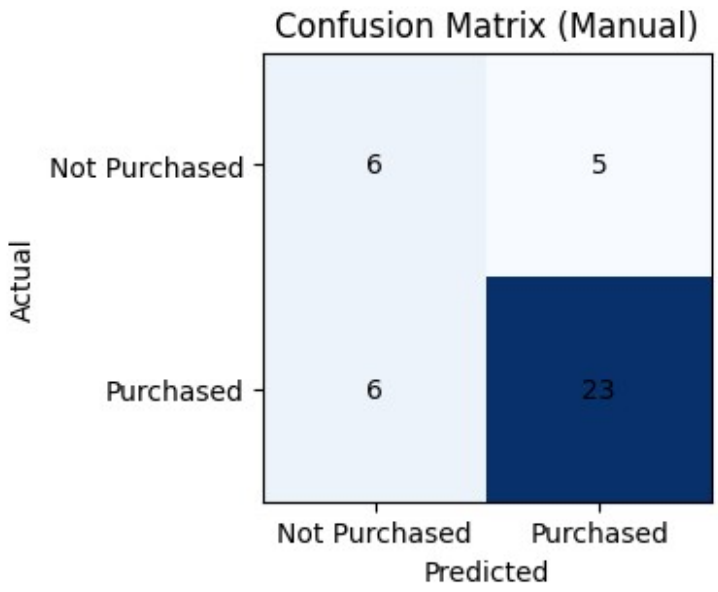
The **F1 score** is the **harmonic mean** of Precision and Recall:

$$F1 = 2 \times \frac{(\text{Precision} \times \text{Recall})}{(\text{Precision} + \text{Recall})}$$

It balances **false positives** and **false negatives** — very useful when your dataset is **imbalanced** (e.g., more non-purchasers than purchasers).

```
import matplotlib.pyplot as plt

# Visualize confusion matrix using matplotlib
plt.figure(figsize=(4,3))
plt.imshow(cm, cmap='Blues')
plt.title('Confusion Matrix (Manual)')
plt.xlabel('Predicted')
plt.ylabel('Actual')
plt.xticks([0,1], ['Not Purchased', 'Purchased'])
plt.yticks([0,1], ['Not Purchased', 'Purchased'])
for i in range(2):
    for j in range(2):
        plt.text(j, i, cm[i,j], ha='center', va='center',
color='black')
plt.show()
```


Confusion Matrix (Manual)
Not Purchased
Purchased
Actual
6
5
6
23
Not Purchased
Purchased
Predicted